libraries

Google colaboratory is equiped with preinstalled all libraries needed.

In [ ]:
```
""" if not installed please use this code"""

%pip install torch numpy pandas scipy statsmodels
```

In [ ]:
```
import itertools
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from statsmodels.stats.diagnostic import lilliefors
```

Architecture

Instructions to follow

The below code is utilized to run the hyperparameter grid for Fixed hyperparameters. The Data Generated from "Data Generating Code.ipynb" can be upoloaded here. The default values for MC_ID = 0, Batch Size = 32 CSV Path should be updated . which is the Data Generated with "CSV" extension Results Path can also be changed accordingly.

In [ ]:
```
# Set random seeds for reproducibility
SEED = 2000
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
np.random.seed(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# ==================== Hyperparameter Grid ====================
HYPERPARAMETER_GRID = {
    'learning_rate': [1e-4, 1e-3],
    'hidden_layers': [2, 4, 6],
    'units_per_layer': [8, 16, 20],
    'activation': ['relu', 'softplus'],
    'beta': [0.0, 0.5, 1.0],
}

# Fixed hyperparameters
REGULARIZATION = 0
EPOCHS = 100
BATCH_SIZE = 32
MC_ID = 9
CSV_PATH = "regression_data_Ishigami.csv"
```

```python
RESULTS_PATH = "results_ishigami.csv"
# ============================================================

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ==================== Data Loading (once outside loop) ====================
def load_data_from_csv(csv_path, mc_id=0, device="cuda"):
    print(f"Loading data from {csv_path} for mc_id={mc_id}...")
    df = pd.read_csv(csv_path)
    df = df[df['mc_id'] == mc_id].copy()

    train_df = df[df['dataset'] == 'train'].copy()
    test_df = df[df['dataset'] == 'test'].copy()

    if 'sample_id' in train_df.columns:
        train_df = train_df.sort_values('sample_id')
        test_df = test_df.sort_values('sample_id')

    X_train = torch.tensor(train_df[['x1', 'x2', 'x3']].values, dtype=torch.floa
    y_train = torch.tensor(train_df['y'].values, dtype=torch.float32, device=dev
    X_test = torch.tensor(test_df[['x1', 'x2', 'x3']].values, dtype=torch.float3
    y_test = torch.tensor(test_df['y'].values, dtype=torch.float32, device=devic

    print(f"Data loaded: n_train={len(X_train)}, n_test={len(X_test)}")
    return X_train, y_train, X_test, y_test


# ==================== Model ====================
class GaussianMLP(nn.Module):
    def __init__(self, input_dim, hidden_sizes, activation='relu'):
        super().__init__()
        activation_map = {
            'relu': nn.ReLU(),
            'softplus': nn.Softplus(),
            'tanh': nn.Tanh()
        }
        act_fn = activation_map[activation.lower()]
        layers = []
        prev = input_dim
        for h in hidden_sizes:
            layers.append(nn.Linear(prev, h))
            layers.append(act_fn)
            prev = h
        self.shared = nn.Sequential(*layers)
        self.head = nn.Linear(prev, 2)

    def forward(self, x):
        out = self.shared(x)
        out = self.head(out)
        return out[:, 0:1], out[:, 1:2]


def beta_nll_loss(mean, variance, target, beta=0.5):
    loss = 0.5 * ((target - mean) ** 2 / variance + variance.log())
    if beta > 0:
        loss = loss * (variance.detach() ** beta)
    return loss.sum(axis=-1).mean()


def train_gaussian(model, train_loader, epochs, lr, weight_decay, beta):
```

```
    model.to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    for epoch in range(epochs):
        model.train()
        epoch_loss, num_batches = 0.0, 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            mean, var_raw = model(x)
            variance = F.softplus(var_raw) + 1e-8
            variance = torch.clamp(variance, max=1000.0)
            loss = beta_nll_loss(mean, variance, y, beta=beta)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
            num_batches += 1
        if epoch % 50 == 0:
            print(f"  Epoch {epoch}/{epochs} - Loss: {epoch_loss/num_batches:.4f


@torch.no_grad()
def evaluate_gaussian(model, X_data, y_data):
    model.eval()
    mean, var_raw = model(X_data)
    var = F.softplus(var_raw) + 1e-8
    var = torch.clamp(var, max=1000.0)
    preds = mean.flatten()
    vars_output = var.flatten()
    rmse = torch.sqrt(((preds - y_data.flatten()) ** 2).mean()).item()
    return rmse, preds, vars_output


def calculate_residuals(preds, y_true):
    return y_true.cpu().flatten() - preds.cpu()


def calculate_rmse_true(preds, X_data, a=7, b=0.1):
    X_cpu = X_data.cpu()
    x1, x2, x3 = X_cpu[:, 0], X_cpu[:, 1], X_cpu[:, 2]
    true_mean = torch.sin(x1) + a * torch.sin(x2)**2 + b * (x3**4) * torch.sin(x
    return torch.sqrt(((preds.cpu() - true_mean)**2).mean()).item()


def lilliefors_test(residuals):
    residuals_np = residuals.numpy() if torch.is_tensor(residuals) else residual
    return lilliefors(residuals_np, dist='norm')


def save_results_to_csv(mc_id, config, metrics, save_path="results_ishigami.csv"
    results = {
        'mc_id': mc_id,
        'learning_rate': config['lr'],
        'hidden_layers': config['hidden_layers'],
        'units_per_layer': config['units'],
        'activation': config['activation'],
        'regularization': config['l2'],
        'beta': config['beta'],
        'batch_size': config['batch_size'],
        'train_rmse_observed': metrics['train_rmse'],
        'train_rmse_true': metrics['train_rmse_true'],
```

```
        'test_rmse_observed': metrics['test_rmse'],
        'test_rmse_true': metrics['test_rmse_true'],
        'train_lilliefors_statistic': metrics['lf_statistic'],
        'train_lilliefors_pvalue': metrics['lf_pvalue'],
        'train_residual_mean': metrics['train_residual_mean'],
        'train_residual_std': metrics['train_residual_std'],
        'train_residual_variance': metrics['train_residual_variance'],
        'test_lilliefors_statistic': metrics['lf_statistic_test'],
        'test_lilliefors_pvalue': metrics['lf_pvalue_test'],
        'test_residual_mean': metrics['test_residual_mean'],
        'test_residual_std': metrics['test_residual_std'],
        'test_residual_variance': metrics['test_residual_variance'],
    }
    df = pd.DataFrame([results])
    try:
        existing_df = pd.read_csv(save_path)
        df = pd.concat([existing_df, df], ignore_index=True)
    except FileNotFoundError:
        pass
    df.to_csv(save_path, index=False)


# ==================== Load Data Once ====================
X_train, y_train, X_test, y_test = load_data_from_csv(
    csv_path=CSV_PATH, mc_id=MC_ID, device=device
)
train_dataset = TensorDataset(X_train, y_train)

# ==================== Hyperparameter Loop ====================
keys = list(HYPERPARAMETER_GRID.keys())
values = list(HYPERPARAMETER_GRID.values())
all_combinations = list(itertools.product(*values))
total = len(all_combinations)

print(f"\nStarting grid search: {total} combinations\n{'='*60}")

for run_idx, combo in enumerate(all_combinations):
    params = dict(zip(keys, combo))

    LR          = params['learning_rate']
    HIDDEN      = params['hidden_layers']
    UNITS       = params['units_per_layer']
    ACTIVATION  = params['activation']
    BETA        = params['beta']

    print(f"\n[{run_idx+1}/{total}] LR={LR}, Hidden={HIDDEN}, Units={UNITS}, "
          f"Activation={ACTIVATION}, Beta={BETA}")

    # Re-seed before each run for reproducibility
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
    np.random.seed(SEED)

    # Build DataLoader (reshuffle each run)
    train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True

    # Build model
    model = GaussianMLP(
        input_dim=3,
        hidden_sizes=[UNITS] * HIDDEN,
```

```python
        activation=ACTIVATION
    )

    # Train
    train_gaussian(
        model, train_loader,
        epochs=EPOCHS, lr=LR,
        weight_decay=REGULARIZATION, beta=BETA
    )

    # Evaluate
    rmse_train, preds_train, _ = evaluate_gaussian(model, X_train, y_train)
    rmse_true_train = calculate_rmse_true(preds_train, X_train)
    rmse_test, preds_test, _ = evaluate_gaussian(model, X_test, y_test)
    rmse_true_test = calculate_rmse_true(preds_test, X_test)

    # Residuals
    residuals_train = calculate_residuals(preds_train, y_train)
    residuals_test = calculate_residuals(preds_test, y_test)

    lf_stat_train, lf_pval_train = lilliefors_test(residuals_train)
    lf_stat_test,  lf_pval_test  = lilliefors_test(residuals_test)

    train_res_np = residuals_train.numpy()
    test_res_np  = residuals_test.numpy()

    # Save
    save_results_to_csv(
        mc_id=MC_ID,
        config={
            'lr': LR, 'hidden_layers': HIDDEN, 'units': UNITS,
            'activation': ACTIVATION, 'l2': REGULARIZATION,
            'beta': BETA, 'batch_size': BATCH_SIZE
        },
        metrics={
            'train_rmse': rmse_train,
            'train_rmse_true': rmse_true_train,
            'test_rmse': rmse_test,
            'test_rmse_true': rmse_true_test,
            'lf_statistic': lf_stat_train,
            'lf_pvalue': lf_pval_train,
            'train_residual_mean': float(train_res_np.mean()),
            'train_residual_std': float(train_res_np.std()),
            'train_residual_variance': float(train_res_np.var()),
            'lf_statistic_test': lf_stat_test,
            'lf_pvalue_test': lf_pval_test,
            'test_residual_mean': float(test_res_np.mean()),
            'test_residual_std': float(test_res_np.std()),
            'test_residual_variance': float(test_res_np.var()),
        },
        save_path=RESULTS_PATH
    )

    print(f"  Train RMSE: {rmse_train:.4f} | Test RMSE: {rmse_test:.4f} | "
          f"LF p-train: {lf_pval_train:.4f} | LF p-test: {lf_pval_test:.4f}")
    print(f"  Saved to {RESULTS_PATH}")

print(f"\n{'='*60}")
print(f"Grid search complete. {total} runs saved to {RESULTS_PATH}")
```